

Sidequest

Obsah

1. Úvod
2. Architektúra a štruktúra kódu
3. Dátový model
4. Algoritmy a kľúčové funkcie
5. Možnosti rozšírenia

1. Úvod

Sidequest je jednoduchý widget na prehľadnú organizáciu a plánovanie úloh počas dňa. Používa SQLite databázu na ukladanie úloh - "questov", ktoré zobrazuje pomocou rozhrania WPF. Celá aplikácia je napísaná v jazyku C# a rozhranie v XAML.

2. Architektúra a štruktúra kódu

Celá aplikácia pracuje v jednom okne, ktoré má oddelené gridy pre hlavné zobrazenie questov (MainGrid) a pre plánovač a jeho nastavenia (SettingsGrid) medzi ktorými má užívateľ možnosť prepínať podľa potreby. Hlavný grid obsahuje 3 zoznamy questov - aktívne, po deadline a dokončené. Rozhranie na pridanie questu a úpravu je rovnaké pre obe akcie.

Sidequest nevyužíva striktný MVVM vzor, ale kvôli priamočiarejšej manipulácii a animácii zväčšenia okna využíva Code-Behind.

3. Dátový model

Sidequest pracuje s dvomi tabuľkami v databáze quests.db:

- **Quests**: obsahuje základné atribúty úloh - ID, názov, poznámka, deadline, časový odhad, flag dokončenia
- **QuestDependencies**: slúži ako väzbová tabuľka na reprezentáciu hrán v grafe závislostí. Obsahuje dvojice ID questov, ktoré definujú, aký quest musí byť dokončený predtým ako sa odomkne iný.

Údaje z tabuliek sú po spustení načítané do operačnej pamäte a spracované ako objekty (pre zoznamy do ObservableCollection, pre závislosti Dictionary) čo zabezpečuje rýchly chod aplikácie.

4. Algoritmy a kľúčové funkcie

Dôležitou funkciou aplikácie je inteligentný plánovač, ktorý využíva princípy teórie grafov na správne a optimálne zoradenie questov počas dňa.

Modelovanie závislostí

Závislosti medzi questami sú reprezentované ako orientovaný acyklický graf (DAG). Pre maximálnu rýchlosť je tento graf udržiavaný v pamäti ako zoznam susedov pomocou štruktúry `Dictionary<int, List<int>>`, čo umožňuje prístup k uzlom v čase $O(1)$.

Detekcia cyklov - DFS

Pri úprave questu a zmene závislostí by užívateľ mohol vytvoriť deadlock - cyklus v grafe, čo by znemožnilo korektné plánovanie. Aby sa tomuto predišlo, využíva sidequest DFS - prehľadávanie do hĺbky na kontrolu acyklickosti grafu. Funkcia `CheckForCycles` tento algoritmus implementuje. Pomocou lokálnej množiny `HashSet<int>` pamätá uzly v aktuálnej vete rekúzie. Ak nájde uzol ktorý už v zásobníku je, ohlásí cyklus a zakáže užívateľovi uložiť quest kým neopraví chybu v závislostiach. Táto operácia sa vykonáva v čase $O(V + E)$, kde V je počet vrcholov v grafe a E počet hrán (závislostí) medzi nimi.

Hodnotiaca funkcia - skóre urgentnosti

Funkcia `EvaluateQuestUrgency` slúži na výpočet priority questov na základe dvoch hlavných metrik:

- Časová rezerva: čas do deadlinu - časový odhad úlohy. Menšia rezerva prioritu zvyšuje exponenciálne. Vďaka funkcii `Math.Max(timeReserve, 0.1)` menovateľ neklesne pod 0.1, čím sa predchádza deleniu nulou.
- Blokovací faktor (out-degree v DAGu): koľko iných questov bezprostredne čaká na dokončenie aktuálneho

$$Score = \frac{1000}{\max(timeReserve, 0.1)} + (neighbors \cdot 50)$$

Plánovanie - upravený Kahnov algoritmus

Pre plánovanie dňa bol vo funkcii `PlanQuests` implementovaný upravený Kahnov algoritmus pre topologické triedenie. Namiesto klasickej fronty využíva max-haldu - prioritnú frontu prostredníctvom triedy `PriorityQueue<Quest, int>`. Algoritmus najprv spočíta in-degree, t.j. počet nesplnených prerekvizít pre každý quest v grafe a úlohy s nulovým in-degree vloží do haldy, ktorá ich zoradí podľa skóre urgentnosti. Potom v každom kroku sa z fronty vyberie prvý (najakútnejší) quest, vloží sa do plánu a ak niektorému z jeho susedov klesne in-degree na nulu, vloží sa do haldy. Plánovanie pracuje v čase $O(V + E \log V)$

Dynamické vkladanie prestávok

Užívateľ si môže v nastaveniach plánovača zvoliť celkový čas práce za deň a počet 20-minútových prestávok počas dňa. Kahnov algoritmus potom vypočíta čas práce bez prestávky. Ak čas nepretržitej práce prekročí tento limit, algoritmus spočíta vzdialenosť od ideálneho času prestávky pred pridaním questu a po pridaní a prestávku vloží tam kde je to výhodnejšie, t.j. bližšie k limitu, čím sa predchádza neλογickému vkladaniu prestávok pred dlhé úlohy.

5. Možnosti rozšírenia

- Prechod na vzor MVVM čo by uľahčilo automatizované testovanie.
- Štatistiky a vizualizácia: sledovanie dát o efektívite užívateľa a dodržiavanie deadlinov a odhadov vo forme grafov
- Úprava plánovania: užívateľ by si mohol vybrať či chce prioritizovať spĺňanie deadlinov alebo odomykať blokové úlohy
- Viac nastavení: napr. farba, poloha na obrazovke, veľkosť,...
- Zdieľanie úloh napr. v rámci lokálnej siete